

NagySoft

Bienvenidos al mundo real ... bueno, por lo menos una aproximación de este.

Béla Szilard, PhD, es un Húngaro-Americano y el dueño de NagySoft, una compañía consultora, para medianas y grandes empresas, que se especializa en resolver problemas que requieren conocimientos de estadística, "machine learning", inteligencia artificial, y computación. Ustedes han sido seleccionados como pasantes de NagySoft. Considérense afortunados: los mejores pasantes reciben ofertas para ser empleados de la compañía, y NagySoft paga bien. Mis roles en la empresa son de liderar el desarrollo de software y reclutar talento. El Dr. Szilard no habla Español - yo me encargo de pasarles sus requerimientos.

Las reglas generales del Dr. Szilard son:

- Pueden aprovechar material en la web, e inteligencia artificial, pero no pedir ayuda a otras personas.
- Deben proveer un artefacto, típicamente un programa, que resuelve o ayuda a resolver el problema.
- Dicho artefacto debe producir resultados presentados en la forma requerida por la empresa.
- El artefacto debe proveer la documentación requerida por la empresa.

Como de costumbre, el Dr. Szilard escogió una tarea que nos permite apreciar su capacidad de:

- buscar información (por ejemplo, "*¿Que es un centroide?*") para entender problemas y buscar soluciones
- usar su inventiva (arreglárselas y desarrollar *sus propias ideas* para superar obstáculos)
- resolver problemas acuerdo a las reglas y especificaciones dadas aquí.

¡Esas son las personas que el Dr. Szilard quiere contratar!

NagySoft los está retando con un problema de clasificación. En esta carpeta van a conseguir un archivo de datos (**datos.csv**) donde cada línea representa un punto en un espacio de 3 dimensiones. La idea es identificar "grupos" (clusters) de puntos que están ... bueno ... "agrupados", es decir cercanos a otros. Vean la imagen "**Clusters.png**" en esta carpeta. Existe un algoritmo, de hecho, una familia de algoritmos, llamados "k-means clustering", que son **muy bien conocidos** y utilizados para atacar esta clase de problemas.

Para aclarar el concepto, la palabra "cluster" (en Inglés) aparece en varios contextos, por ejemplo:

- Un "Galaxy Cluster" es un Cúmulo de Galaxias.
- Un "Computing Cluster" es un conjunto de servidores (computadoras) que ofrecen mejor confiabilidad - por tener redundancia - y mejor rendimiento - más computación en paralelo - que una sola computadora.
- Un "Grape Cluster" es un racimo de uvas.

En calidad de **sustantivo**, la palabra "cluster" puede traducirse como "grupo", "agrupamiento", "cúmulo", ... En calidad de **verbo** la palabra "cluster" significa "agrupar", "juntar", "clasificar", ...

De ahora en adelante voy a usar la palabra "cluster" en el contexto de este problema ... ¡Cómo de hecho lo hacen la mayoría de los ingenieros de habla hispana!

Para los estudiantes Pythonicos, les tengo buenas noticias:

- van a conseguir con facilidad ejemplos de "k-means clustering"
 - aún más, pueden conseguir una función en las librerías que implementa el "k-means clustering" ...
- ... ya los escucho decir: ¡¡¡OMG, el problema está resuelto!!!

Calmen la "viveza criolla" hasta llegar al final.

Lo anterior puede ayudarlos, como referencia, pero aun así van a tener que trabajar.

Aquí les damos recursos (en G-Drive por ahora)

URLs: en la carpeta van a conseguir URLs para:

- **distancia Euclidiana** entre dos puntos, en múltiples dimensiones

- **k-means clustering** (Wikipedia) y k-means clustering (Stanford CS221)

La página de k-means clustering de Wikipedia puede intimidarlos, pero sugiero que estudien el pseudocódigo, es bastante bueno.

La página de Stanford es más amigable para estudiantes - lean el pseudocódigo Pythonico: buena modularización y la condición de parada encapsulada en una función, como lo pidió el Dr. Szilard.

Cargador de archivo:

En la carpeta C++/5.Tablas van a ver dos ejemplos de cómo cargar un archivo de datos en un vector.

El Dr. Szilard es sumamente **estricto** con las reglas y **exigente** acerca de la calidad del software.

El VP de R&D (ese soy yo) también lo es, pero tiene más paciencia con los pasantes.

Lean las instrucciones con cuidado y respeten todos los requerimientos.

No abusen mi tiempo obligándome a adaptar su programa para ver o comprobar los resultados.

Requerimientos:

- El artefacto debe ser implementado completamente en **C++**.

- El artefacto debe ser **modular**: el programa principal debe diferir a un módulo para el cargador de datos, otro para el algoritmo y otros módulos.

- En general, es buena práctica que la entrada (carga de datos) y la salida se realicen en *módulos separados*

- Utilicen interfaces y referencias (ver 5.Tablas) para acceder a los módulos que contienen la información.

- La condición de parada del programa debe encapsularse en una función, con comentario en el código explicando los parámetros y su explicación de porque es una buena condición.

- El ejecutable debe llamarse **cluster** o **cluster.exe** y se invoca de esta manera: **cluster <k> <datos>** ...

- ... donde <k> es el número de clusters y <datos> el nombre del archivo.

- Noten que **cluster <k> <datos>** se lee: "**agrupa** en <k> clusters los <datos>" ... 😎 ¡cluster como verbo!

- A **priori nunca se sabe** cuántos clusters hay en un conjunto. Deben aplicar "k-means clustering", experimentando para varios valores de **k** (siempre usen k para indicar el número de clusters)

- Para hacerle la vida más fácil, el Dr. Szilard les anticipa que no van a necesitar más de seis clusters.

- Pero no caigan en la trampa de "overfitting" (busquen el significado general en "machine learning"): queremos una clasificación honesta, con sentido común, que no exagere la bondad de la clasificación.

- Una clasificación con más clusters de los necesarios va a darles la ilusión que tienen un mejor resultado, pero es un engaño que no corresponde a la realidad.

- Recomendando que usen una medida para evaluar la dispersión de cada cluster relativamente a su centroide y ver cuándo al aumentar **k** el resultado no cambia de manera significativa.

- ¿Qué medida? Hay muchas medidas de dispersión en estadística ... pueden inventar una (yo lo he hecho) o buscar una apropiada en la literatura.

- Los puntos de los datos deben representarse con esta estructura:

```
struct Coord_3D {
    double x;
    double y;
    double z;
};
```

- El cargador debe poner todos los datos en un **vector<Coord_3D>** el cual no debe ser modificado.
- A cada cluster se le debe asociar un nombre (etiqueta) de **una letra**, usando estas: A, B, C, D, E, F
- Un punto etiquetado (es decir asociado a un cluster) *puede* representarse de esta manera:

```
struct Labeled {
    Coord_3D coord;
    char label;
};
```

- Pero no es mandatorio hacerlo de esa manera: hay formas más prácticas, dependiendo del algoritmo.
- Aun así, **van a tener que etiquetarlos** con 'A' .. 'F' en la salida (vean el ejemplo reducido **clasificados.csv**)
- El programa *debe* generar un archivo (**clasificados.csv**) con todos los puntos etiquetados.
- No es necesario que los puntos en **clasificados.csv** tengan el mismo orden de **datos.csv**.
- El programa también debe generar un archivo (**summary.txt**) como este ejemplo (molde):

A: N , (x, y, z) , MD
 B: N , (x, y, z) , MD
 C: N , (x, y, z) , MD

- En el ejemplo, N es el número de puntos, (x, y, z) las coordenadas del centroide, y MD la medida de dispersión, del cluster correspondiente. Noten que el ejemplo *asume* que tenemos 3 clusters.
- Sólo deben presentar un summary.txt, con el numero de clusters Uds. Consideran adecuado.

Por último (espero no haber olvidado nada) quiero aclarar y reiterar:

- Pueden utilizar inteligencia artificial (incluso varios modelos) y fuentes de la Web, libros, blogs, etc.
- Pero no deben tener ayuda de otras personas.
- En su repositorio, deben tener una carpeta, con uno o más archivos indicando las **fuentes**, mostrando **imágenes**, y reportando sus **comentarios**.
- Entre ellos documenten todas las interacciones con inteligencia artificial.
- Usar inteligencia artificial **NO** va a afectar su nota. De hecho, alentamos su uso en este examen. Esto es lo que Uds. quieren, y lo estamos apoyando.
- Las interacciones con IA (y sus comentarios, de tenerlos) nos van a ayudar a realizar investigación de enseñanza y entender cómo les ayudó o lo contrario.
- Pero, si detectamos plagiarismo entre personas, los involucrados van a ser botados de la pasantía (imagínense lo que eso significa en nuestra vida real)

El "*k-means clustering*" es uno de los algoritmos más conocidos y sencillos de Machine Learning. Lo sé por mi propia experiencia: tomé una suite de cursos (Coursera) de Machine Learning: 2012-2013. Implemente el "*k-means clustering*" en C++ después de haberlo hecho en MATLAB, aunque hacerlo en C++ no era parte de la tarea. Debo tener ese código en algún sitio. Si esto les parece difícil es porque casi todo en la vida parece complicado antes de entenderlo, pero el "*k-means clustering*" no es difícil de entender.